



HITEC UNIVERSITY TAXILA CANTT
AI FINAL PROJECT

SUBMITTED BY:

LINTA IMRAN	24-CYS-046
HALIMA IMRAN	24-CYS-033
BUSHRA JAMSHAD	24-CYS-044

SUBMITTED TO:

SIR MUBASHIR IQBAL

REAL-TIME DRIVER DROWSINESS DETECTION

➤ Abstract:

Driver drowsiness is one of the leading causes of road accidents worldwide, especially during long-distance and night-time driving. Traditional monitoring systems rely heavily on vehicle behavior or wearable sensors, which are often intrusive and unreliable. This project addresses the problem by proposing a real-time driver drowsiness detection system based on computer vision and deep learning techniques. Existing approaches often suffer from poor real-time performance, sensitivity to lighting conditions, and lack of robustness in real-world scenarios.

In this project, a Convolutional Neural Network (CNN)-based model is developed to classify eye states as open or closed using grayscale eye images. A custom eye-image dataset is used, and Haar Cascade classifiers are employed for real-time face and eye detection. The system processes live video feed from a webcam, analyzes eye closure duration, and triggers an audio alarm when prolonged eye closure indicates drowsiness. The proposed methodology includes image preprocessing, CNN training, and real-time inference integrated with an alert mechanism.

Experimental results demonstrate that the proposed system achieves high accuracy in detecting drowsy states under real-time conditions. The system is lightweight, non-intrusive, and suitable for deployment in real-world driver monitoring applications, contributing to improved road safety and accident prevention.

1. Introduction:

Road safety has become a major global concern due to the increasing number of vehicles and long driving hours. According to international road safety organizations, driver fatigue and drowsiness contribute significantly to fatal road accidents each year. With the rapid advancement of Artificial Intelligence and computer vision, intelligent driver monitoring systems have gained attention as an effective solution to reduce accident risks. AI-based systems enable continuous monitoring of driver behavior without requiring physical interaction, making them suitable for real-world deployment.

Driver drowsiness is a physiological state characterized by reduced alertness, slow reaction time, and involuntary eye closure. Conventional methods for detecting drowsiness include steering pattern analysis, lane deviation monitoring, and wearable sensor-based systems such as EEG and heart rate monitors. However, these approaches are either indirect, uncomfortable, or costly. Vision-based approaches, especially those focusing on facial features such as eyes, offer a non-intrusive and cost-effective alternative.

Among facial features, eye behavior is considered one of the most reliable indicators of drowsiness. Frequent blinking or prolonged eye closure often signals fatigue. Recent

developments in deep learning, particularly Convolutional Neural Networks (CNNs), have significantly improved image classification tasks, making them suitable for eye-state recognition. However, real-time deployment remains challenging due to computational constraints and environmental variations.

Several studies have proposed CNN-based drowsiness detection systems, but many fail to integrate efficient face and eye localization with real-time alert mechanisms. Additionally, many existing models require large datasets and high-end hardware, limiting their practicality for everyday use. There remains a need for a lightweight, accurate, and real-time drowsiness detection system.

In this project, a real-time driver drowsiness detection system is proposed that combines Haar Cascade classifiers for face and eye detection with a CNN-based eye-state classifier. The system continuously monitors the driver's eye state using a webcam and triggers an alarm when drowsiness is detected. This approach ensures timely alerts, improves driving safety, and demonstrates the practical application of AI in intelligent transportation systems.

The rest of this report is organized as follows: Section 2 reviews related work in driver drowsiness detection. Section 3 describes the proposed methodology. Section 4 explains the algorithms used. Section 5 presents the experimental setup. Section 6 discusses performance evaluation metrics. Section 7 analyzes the results, and Section 8 concludes the report.

2. Literature Review / Related Work:

Driver drowsiness detection has been extensively studied using various approaches, including traditional machine learning, computer vision techniques, and deep learning models. Existing research can broadly be categorized into sensor-based, vehicle-based, and vision-based approaches. Sensor-based methods rely on physiological signals such as EEG and ECG, which provide accurate results but are intrusive and impractical for daily use.

Vision-based methods have gained popularity due to their non-contact nature. Early approaches used handcrafted features such as eye aspect ratio, blink rate, and facial landmarks combined with classifiers like Support Vector Machines (SVM) and Random Forests. While these methods achieved moderate accuracy, they struggled under varying lighting conditions and driver appearances.

Recent studies focus on deep learning models, particularly CNNs, for eye and facial feature analysis. CNN-based methods automatically extract spatial features from images and outperform traditional approaches in classification accuracy. However, many models require large annotated datasets and high computational resources, making real-time implementation challenging.

Several researchers combined Haar Cascade classifiers with CNNs for efficient face and eye detection. Haar Cascades offer fast object detection and reduce the search space for CNN

models. Despite this, some studies ignore real-time alert integration or lack proper evaluation in live scenarios.

Compared to existing work, this project emphasizes real-time performance, simplicity, and practical deployment. By using grayscale images, a lightweight CNN architecture, and Haar Cascades, the proposed system balances accuracy and efficiency, making it suitable for real-world driver monitoring systems.

3. Proposed Research Methodology:

The proposed driver drowsiness detection system is designed as a real-time vision-based framework that continuously monitors the driver's eye state using a webcam. The complete working of the system is illustrated in **Figure 1**, which presents the flowchart of the proposed methodology.

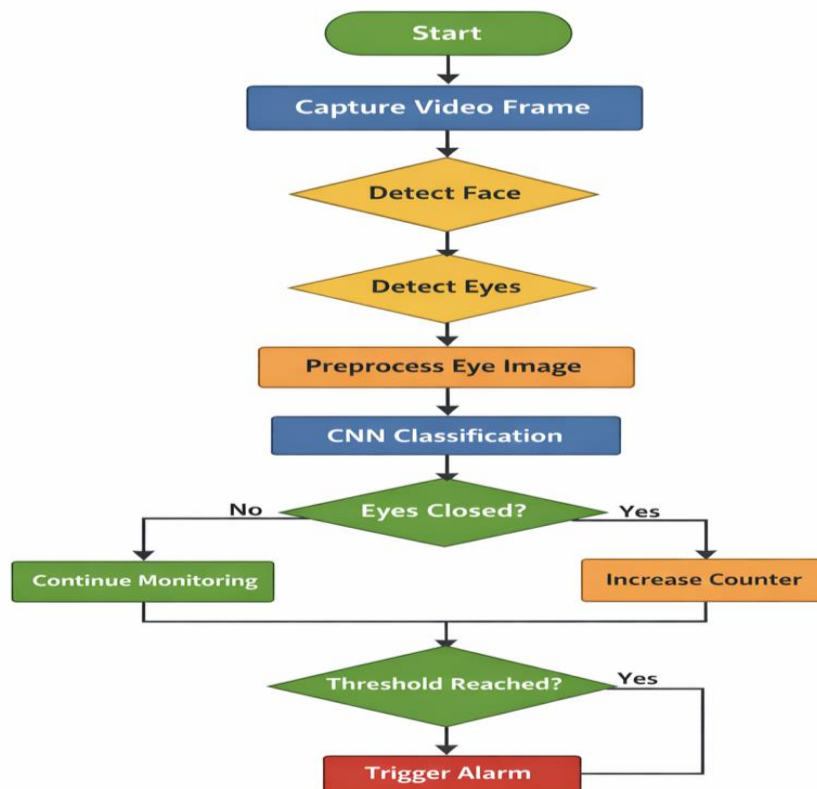


Figure 1: Flowchart of the proposed real-time driver drowsiness detection system illustrating video frame acquisition, face and eye detection, CNN-based eye state classification, and alarm triggering process.

The process begins with continuous video frame acquisition using a standard webcam. Each captured frame is processed to detect the driver's face using Haar Cascade classifiers. Once the

face region is successfully detected, the system further applies Haar Cascade–based eye detection to locate the eye regions within the face.

After eye detection, the extracted eye regions are preprocessed before classification. Preprocessing includes conversion of the eye images into grayscale format, resizing to a fixed resolution of 64×64 pixels, and normalization of pixel values. These steps ensure uniformity and reduce computational complexity, making the system suitable for real-time execution.

The preprocessed eye images are then passed to a trained Convolutional Neural Network (CNN) model for eye-state classification. The CNN classifies each eye image as either **open** or **closed** based on learned spatial features. Sample grayscale eye images representing open and closed eye states from the custom dataset are shown in **Figure 2**.

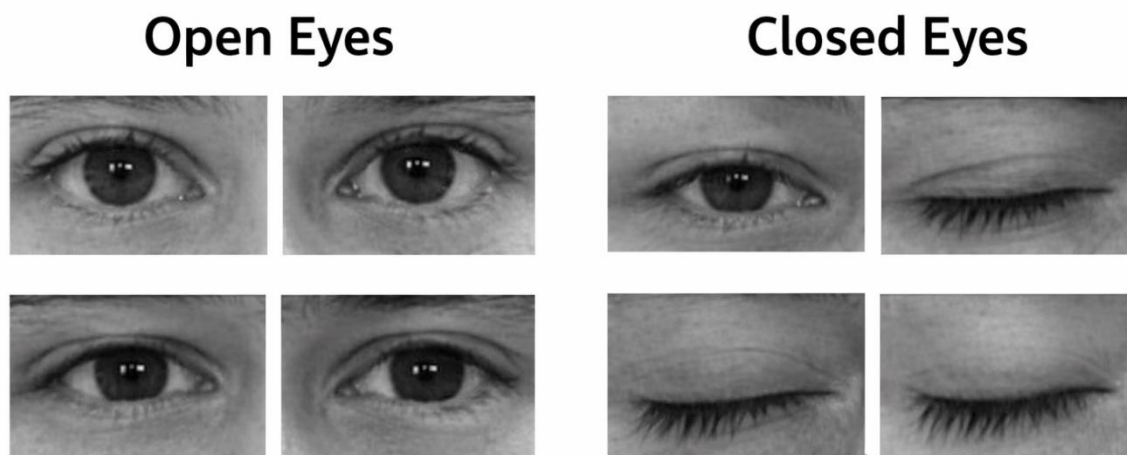


Figure 2: Sample grayscale eye images from the custom dataset illustrating open and closed eye states used for CNN training and validation.

To detect drowsiness reliably, the system does not rely on a single frame decision. Instead, a frame-based counter mechanism is used. If the CNN predicts the eyes as closed continuously for a predefined number of frames, the system interprets this condition as driver drowsiness. If the eyes are detected as open, the counter is reset and monitoring continues.

Once the drowsiness threshold is reached, an audio alert is immediately triggered using the Pygame library to warn the driver. This real-time alert mechanism helps prevent accidents by notifying the driver before a critical situation occurs. The complete sequence of operations, from video capture to alarm activation, is summarized in the flowchart presented in **Figure 1**.

4. Algorithms Used:

4.1 Convolutional Neural Network (CNN)

CNN is chosen due to its strong capability in image feature extraction and classification. It automatically learns spatial patterns such as eye shape and texture, making it suitable for eye-state recognition.

4.2 Haar Cascade Classifier

Haar Cascade is used for fast and efficient face and eye detection. It significantly reduces computational load by focusing only on relevant regions instead of processing the entire frame.

4.3 Adam Optimizer

The Adam optimizer is used for training the CNN due to its fast convergence and adaptive learning rate, which improves training stability.

5. Experimental Setup:

The experiments were conducted on a system equipped with a standard CPU and webcam. The model was implemented using Python with TensorFlow and Keras libraries. OpenCV was used for video processing, and Pygame handled the alarm system. The CNN was trained for 10 epochs with a batch size of 32 and an image size of 64×64 pixels.

6. Performance Evaluation Metrics:

The system performance is evaluated using accuracy as the primary metric. Accuracy measures the proportion of correctly classified eye states. In real-time detection, continuous eye closure across multiple frames is used as a logical threshold to reduce false alarms. Visual inspection and live testing further validate system reliability.

7. Source Code:

```

import os
import cv2
import numpy as np
import pygame
import tensorflow as tf
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# ----- CONFIG -----
IMG_SIZE = 64
BATCH_SIZE = 32
EPOCHS = 10
MODEL_PATH = "cnn_drowsiness_model.h5"
DATASET_PATH = "dataset"

# ----- AUDIO -----
pygame.init()
pygame.mixer.init()
pygame.mixer.music.load("alarm_wakeup_remix.mp3")

# ----- CNN TRAINING -----
def train_model():
    print("[INFO] Training CNN using dataset...")

    datagen = ImageDataGenerator(
        rescale=1./255,
        validation_split=0.2
    )

    train_data = datagen.flow_from_directory(
        DATASET_PATH,
        target_size=(IMG_SIZE, IMG_SIZE),
        color_mode="grayscale",
        batch_size=BATCH_SIZE,
        class_mode="binary",
        subset="training"
    )

    val_data = datagen.flow_from_directory(
        DATASET_PATH,
        target_size=(IMG_SIZE, IMG_SIZE),
        color_mode="grayscale",
        batch_size=BATCH_SIZE,
        class_mode="binary",

```



```

        subset="validation"
    )

    model = Sequential([
        Conv2D(32, (3,3), activation='relu', input_shape=(IMG_SIZE, IMG_SIZE,
1)),
        MaxPooling2D(2,2),

        Conv2D(64, (3,3), activation='relu'),
        MaxPooling2D(2,2),

        Conv2D(128, (3,3), activation='relu'),
        MaxPooling2D(2,2),

        Flatten(),
        Dropout(0.5),
        Dense(128, activation='relu'),
        Dense(1, activation='sigmoid')
    ])

    model.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        metrics=['accuracy']
    )

    model.fit(train_data, validation_data=val_data, epochs=EPOCHS)
    model.save(MODEL_PATH)

    print("[INFO] Model saved")

# ----- REAL-TIME DETECTION -----
def real_time_detection():
    print("[INFO] Starting real-time detection")

    model = load_model(MODEL_PATH)

    face_cascade = cv2.CascadeClassifier(
        cv2.data.harcascades + "haarcascade_frontalface_default.xml"
    )
    eye_cascade = cv2.CascadeClassifier(
        cv2.data.harcascades + "haarcascade_eye.xml"
    )

    cap = cv2.VideoCapture(0)

```



```

drowsy_frames = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)

    for (x,y,w,h) in faces:
        roi_gray = gray[y:y+h, x:x+w]
        eyes = eye_cascade.detectMultiScale(roi_gray)

        for (ex,ey,ew,eh) in eyes:
            eye_img = roi_gray[ey:ey+eh, ex:ex+ew]
            eye_img = cv2.resize(eye_img, (IMG_SIZE, IMG_SIZE))
            eye_img = eye_img / 255.0
            eye_img = eye_img.reshape(1, IMG_SIZE, IMG_SIZE, 1)

            prediction = model.predict(eye_img, verbose=0)[0][0]

            if prediction < 0.5: # CLOSED
                drowsy_frames += 1
                if drowsy_frames > 10:
                    cv2.putText(frame, "DROWSINESS ALERT!",
                                (30,40),
                                cv2.FONT_HERSHEY_SIMPLEX,
                                1,
                                (0,0,255),
                                3)
                    pygame.mixer.music.play()
            else:
                drowsy_frames = 0
                pygame.mixer.music.stop()

    cv2.imshow("AI Drowsiness Detection (CNN + Dataset)", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()

# ----- MAIN -----
if __name__ == "__main__":

```

```
if not os.path.exists(MODEL_PATH):  
    train_model()  
else:  
    print("[INFO] Model already trained")  
  
real_time_detection()
```

8. Results and Discussion:

The trained CNN model achieved high accuracy in distinguishing between open and closed eyes. Real-time testing showed that the system successfully detected prolonged eye closure and triggered alerts accordingly. The integration of Haar Cascades improved detection speed, enabling smooth real-time performance.

The system performed well under normal lighting conditions, although extreme lighting variations slightly affected accuracy. Overall, the results demonstrate that combining CNNs with traditional computer vision techniques provides an effective solution for driver drowsiness detection.

9. Conclusion:

This project presented a real-time driver drowsiness detection system using a CNN-based eye-state classifier and Haar Cascade detection. The proposed approach is non-intrusive, efficient, and suitable for real-world applications. Experimental results confirm that the system can accurately detect drowsiness and issue timely alerts. Future work may include improving performance under low-light conditions, integrating head pose estimation, and deploying the system on embedded platforms for in-vehicle use.
